Model Registry: Log, Deploy, and Serve R Models

The Snowflake Model Registry lets you version, manage, and deploy ML models directly in Snowflake. `snowflakeR` makes this accessible from R – you train models in R, and the package handles the Python bridging behind the scenes.

How it works

When you call `sfr_log_model()`, snowflakeR:

1. Saves your R model to an `.rds` file
2. Auto-generates a Python `CustomModel` wrapper that uses `rpy2` to load and call your R model
3. Registers the wrapped model in Snowflake's Model Registry
4. At inference time (in SPCS), the wrapper loads R, restores the model, and runs your predict function

You never write Python – the package handles everything.

Setup

```r
library(snowflakeR)

conn <- sfr_connect()

# Optional: target a specific db/schema for the registry
reg <- sfr_model_registry(conn, database = "ML_DB", schema = "MODELS")
```

You can pass either `reg` or `conn` as the first argument to all registry functions. Using `conn` directly uses the session's current database/schema.

Train a model

Train any R model as you normally would:

```r
# Simple linear model
model <- lm(mpg ~ wt + hp + cyl, data = mtcars)
summary(model)
```

Test locally before registering

`sfr_predict_local()` runs the exact same prediction logic that will execute inside Snowflake, but entirely in R (no Python bridge):

```r
test_data <- data.frame(wt = c(2.5, 3.0, 3.5), hp = c(110, 150, 200), cyl = c(4, 6, 8))

preds <- sfr_predict_local(model, test_data)
preds
#>   prediction
#> 1   24.46
#> 2   20.12
#> 3   15.18
```

Log the model to Snowflake

```r
mv <- sfr_log_model(
  reg,
  model       = model,
  model_name  = "MTCARS_MPG",
  input_cols  = list(wt = "double", hp = "double", cyl = "integer"),
  output_cols = list(prediction = "double"),
  comment     = "Linear regression predicting MPG from weight, horsepower, cylinders"
)

mv
#> <sfr_model_version>
#>   model: "MTCARS_MPG"
#>   version: "V1"
```

Key parameters

Parameter	Description
`model`	Any R object that can be `saveRDS()`'d
`model_name`	Registry name (uppercase recommended)
`input_cols`	Named list: column name -> type (`"double"`, `"integer"`, `"string"`, `"boolean"`)
`output_cols`	Named list: output column name -> type
`predict_fn`	R function name (default: `"predict"`)
`predict_pkgs`	R packages needed at inference time (e.g., `c("forecast", "xgboost")`)
`conda_deps`	Additional conda packages (r-base and rpy2 are always included)
`target_platforms`	`"SNOWPARK_CONTAINER_SERVICES"` (default) or `"WAREHOUSE"`

Custom prediction code

Some models (e.g., `bsts`, `forecast`) have non-standard `predict()` signatures or return structures that need reshaping. For these, provide a `predict_body` – custom R code executed inside the SPCS container at inference time.

Recommended approach: `sfr_predict_body()`

Write your predict logic as a normal R function, then let `sfr_predict_body()` convert it to the required template format:

```r
# Define inference logic as a testable R function
my_forecast <- function(model, input) {
  pred   <- forecast::forecast(model, h = nrow(input))
  result <- data.frame(
    period         = seq_len(nrow(input)),
    point_forecast = as.numeric(pred$mean),
    lower_95       = as.numeric(pred$lower[, 2]),
    upper_95       = as.numeric(pred$upper[, 2])
  )
}
```

```r
# Convert to template and register
mv <- sfr_log_model(
  reg,
  model        = arima_model,
  model_name   = "SALES_FORECAST",
  predict_pkgs = c("forecast"),
  predict_body = sfr_predict_body(my_forecast),
  input_cols   = list(period = "integer"),
  output_cols  = list(
    period = "integer", point_forecast = "double",
    lower_95 = "double", upper_95 = "double"
  )
)
```

`sfr_predict_body()` replaces the first formal with `{{MODEL}}`, the second with `{{INPUT}}`, and suffixes local variables with `{{UID}}` to prevent name collisions when multiple models run in the same container.

Alternative: raw template strings

You can also write the template directly (useful for very short snippets):

```r
predict_body = paste(
  'pred_{{UID}} <- forecast::forecast({{MODEL}}, h = nrow({{INPUT}}))',
  'result_{{UID}} <- data.frame(',
  '  period = seq_len(nrow({{INPUT}})),',
  '  point_forecast = as.numeric(pred_{{UID}}$mean)',
  ')',
  sep = '\n'
)
```

Template variables:

- `{{MODEL}}` – the deserialized R model object (loaded from RDS)
- `{{INPUT}}` – the input data.frame passed to predict
- `{{UID}}` – unique ID for variable naming (prevents collisions)
- `{{N}}` – number of rows in input
- The code must produce a `result_{{UID}}` data.frame matching `output_cols`

ML Lineage: Feature View -> Dataset -> Model

When training data comes from the Feature Store, you can create an immutable, versioned **Dataset** and pass it to `sfr_log_model()` to complete the full ML Lineage chain visible in Snowsight:

Source Table -> Feature View -> Dataset -> Model

```r
# 1. Generate a Dataset (instead of sfr_generate_training_data)
training <- sfr_generate_dataset(
  fs,
  name     = "CHURN_TRAINING",
  spine    = "SELECT customer_id, label FROM labels",
  features = list(
    list(name = "CUSTOMER_FEATURES", version = "v1")
  ),
  version  = "v1",
  spine_label_cols = "label"
)
```

```r
# training is a plain data.frame with dataset_name/dataset_version attributes
str(training)

# 2. Train your model on the data.frame as usual
model <- glm(label ~ ., data = training, family = "binomial")

# 3. Log the model with lineage
mv <- sfr_log_model(
  reg,
  model            = model,
  model_name       = "CHURN_MODEL",
  input_cols       = sfr_input_cols(training, exclude = "label"),
  output_cols      = list(prediction = "double"),
  training_dataset = training
)
```

The `training_dataset` parameter tells `sfr_log_model()` to use the Dataset-backed Snowpark DataFrame as `sample_input_data`, which Snowflake traces back through the Dataset to its source Feature Views and tables.

Manage models

```r
# List all models
sfr_show_models(reg)

# Get a specific model
m <- sfr_get_model(reg, "MTCARS_MPG")
m
#> <sfr_model> "MTCARS_MPG"
#>   versions: "V1"
#>   default: "V1"

# Show versions
sfr_show_model_versions(reg, "MTCARS_MPG")

# Get a specific version
mv <- sfr_get_model_version(reg, "MTCARS_MPG", "V1")
```

Metrics

Attach evaluation metrics to model versions for tracking and comparison:

```r
# Set metrics
sfr_set_model_metric(reg, "MTCARS_MPG", "V1", "rmse", 2.45)
sfr_set_model_metric(reg, "MTCARS_MPG", "V1", "r_squared", 0.87)

# Retrieve metrics
sfr_show_model_metrics(reg, "MTCARS_MPG", "V1")
#> $rmse
#> [1] 2.45
#> $r_squared
#> [1] 0.87
```

Version management

```r
# Log a new version of the same model
model_v2 <- lm(mpg ~ wt + hp + cyl + disp, data = mtcars)

mv2 <- sfr_log_model(
  reg,
  model        = model_v2,
  model_name   = "MTCARS_MPG",
  version_name = "V2",
  input_cols   = list(wt = "double", hp = "double", cyl = "integer", disp = "double"),
  output_cols  = list(prediction = "double"),
  comment      = "V2: added displacement"
)

# Set default version
sfr_set_default_model_version(reg, "MTCARS_MPG", "V2")
```

Aliases

Aliases provide human-readable labels for model versions, independent of version numbers:

```r
sfr_set_model_alias(reg, "MTCARS_MPG", "V2", "production")
sfr_set_model_alias(reg, "MTCARS_MPG", "V1", "staging")

# Remove an alias
sfr_unset_model_alias(reg, "MTCARS_MPG", "production")
```

Aliases appear in `sfr_show_model_versions()` output and can be used with `sfr_predict_sql()` for warehouse-native inference.

Model introspection

Inspect a model version's metadata without running inference:

```r
# List callable functions (e.g., predict)
sfr_show_model_functions(reg, "MTCARS_MPG", "V1")

# Get or set the version description
sfr_model_description(reg, "MTCARS_MPG", "V1")
sfr_model_description(reg, "MTCARS_MPG", "V1", desc = "Updated LM model")

# Get the task type (if set during logging)
sfr_get_model_task(reg, "MTCARS_MPG", "V1")
```

Granular metric management

In addition to `sfr_set_model_metric()` and `sfr_show_model_metrics()`, you can read and delete individual metrics:

```r
sfr_set_model_metric(reg, "MTCARS_MPG", "V1", "rmse", 2.45)
sfr_set_model_metric(reg, "MTCARS_MPG", "V1", "r_squared", 0.87)

# Get a single metric by name
sfr_get_model_metric(reg, "MTCARS_MPG", "V1", "rmse")
#> [1] 2.45
```

```r
# Delete a metric
sfr_delete_model_metric(reg, "MTCARS_MPG", "V1", "rmse")
```

Version and model management

```r
# Delete a specific version (must not be the default)
sfr_set_default_model_version(reg, "MTCARS_MPG", "V2")
sfr_delete_model_version(reg, "MTCARS_MPG", "V1")

# List Model objects with metadata
sfr_models(reg)
#>             name           comment default_version
#> 1  MTCARS_MPG   Linear regression              V2
#> 2  CHURN_MODEL  GLM churn model                V1
```

Model lineage and export

Trace upstream data sources and downstream consumers, or export model files to a local directory:

```r
# Lineage (requires model logged with sample_input_data/training_dataset)
sfr_model_lineage(reg, "MTCARS_MPG", "V2", direction = "upstream")

# Export model artifacts to a local path
sfr_export_model(reg, "MTCARS_MPG", "V2",
                 target_path = "/tmp/mtcars_export")
```

Remote inference (SPCS)

Once deployed, run inference directly in Snowflake:

```r
# Predict using a Snowpark DataFrame (runs in Snowflake, not locally)
new_data <- sfr_query(conn, "SELECT wt, hp, cyl FROM car_data LIMIT 100")
preds <- sfr_predict(reg, "MTCARS_MPG", new_data)
```

Advanced predict parameters

```r
preds <- sfr_predict(
  reg, "MTCARS_MPG", new_data,
  partition_column = "REGION",           # partition for parallel inference
  strict_input_validation = TRUE         # enforce schema validation
)
```

SQL-direct inference (warehouse)

For models logged with `target_platforms = "WAREHOUSE"`, run inference purely in SQL without SPCS:

```r
sfr_predict_sql(
  conn,
  model_name   = "PYTHON_MODEL",
  version_name = "V1",
  source_table = "SCORING_DATA",
  target_table = "PREDICTIONS"
)
```

This generates and executes `CREATE TABLE ... AS SELECT *, MODEL()!PREDICT(...) FROM source_table`.

Deploy as an SPCS service

For real-time inference endpoints:

```
sfr_deploy_model(
  reg,
  model_name   = "MTCARS_MPG",
  version_name = "V2",
  service_name = "mpg_service",
  compute_pool = "ML_POOL",
  image_repo   = "my_db.my_schema.my_repo"
)

# Predict via the service
preds <- sfr_predict(
  reg, "MTCARS_MPG", new_data,
  service_name = "mpg_service"
)

# List active services for a version
sfr_list_services(reg, "MTCARS_MPG", "V2")

# Clean up
sfr_undeploy_model(reg, "MTCARS_MPG", "V2", "mpg_service")
```

Advanced deployment parameters

`sfr_deploy_model()` supports fine-grained resource control:

```
sfr_deploy_model(
  reg,
  model_name   = "MTCARS_MPG",
  version_name = "V2",
  service_name = "mpg_service_gpu",
  compute_pool = "GPU_POOL",
  image_repo   = "my_db.my_schema.my_repo",
  cpu_requests     = "2",
  memory_requests  = "4Gi",
  gpu_requests     = "1",
  num_workers      = 2,
  max_batch_rows   = 1000,
  max_instances    = 3,
  min_instances    = 1,
  block            = TRUE,
  autocapture      = TRUE,
  image_build_compute_pool = "BUILD_POOL",
  build_external_access_integrations = c("MY_EAI")
)
```

Batch inference (SPCS)

For large-scale scoring jobs that don't need a persistent service:

```r
results <- sfr_run_batch(
  reg,
  model_name    = "MTCARS_MPG",
  version_name  = "V2",
  new_data      = scoring_data,
  compute_pool  = "ML_POOL",
  function_name = "predict"
)
```

Advanced log_model parameters

`sfr_log_model()` supports additional parameters for advanced use cases:

Parameter	Description
`task`	ML task type: `"TABULAR_REGRESSION"`, `"TABULAR_BINARY_CLASSIFICATION"`, etc.
`python_version`	Pin the Python version in the container (e.g., `"3.11"`)
`user_files`	Additional files to include in the model package
`code_paths`	Python source files to include for custom inference code
`resource_constraint`	Resource requirements as a named list (e.g., `list(memory = "4Gi")`)

```r
mv <- sfr_log_model(
  reg,
  model        = model,
  model_name   = "CHURN_MODEL",
  task         = "TABULAR_BINARY_CLASSIFICATION",
  python_version = "3.11",
  input_cols   = list(score = "double", tenure = "integer"),
  output_cols  = list(prediction = "double")
)
```

Clean up

```r
sfr_delete_model(reg, "MTCARS_MPG")
```

Container package versions (important)

When `sfr_deploy_model()` or `create_service()` builds the SPCS container image, it resolves conda/pip package versions at build time. R model containers include `r-base` and `rpy2`, which can affect which versions of numpy, pandas, and Python itself get installed.

Key points:

- Without a `numpy<2.0` pin, the conda solver picks **Python 3.12 + numpy 2.x** for R model containers. Pure Python models get **Python 3.11** + numpy 2.x (which works fine).
- Under Python 3.12 + numpy 2.x the SPCS inference server's JSON deserialisation produces a `numpy.recarray` instead of a `pandas.DataFrame`. This causes an HTTP 500 crash: `recarray has no attribute fillna`.
- Pinning `numpy<2.0` also causes the solver to select **Python 3.11** (the same version used by pure-Python containers), which completely avoids the bug. Our testing confirmed: Python 3.11.14 + numpy 1.26.4.
- `snowflakeR` includes `numpy<2.0` in the default `conda_deps`. If you override `conda_deps`, **make sure to include `numpy<2.0`**.

```r
# Default (safe) -- numpy<2.0 is pinned automatically:
mv <- sfr_log_model(conn, model, model_name = "MY_MODEL",
                    input_cols = list(x = "double"),
                    output_cols = list(prediction = "double"))

# Custom conda deps -- remember to include the numpy pin:
mv <- sfr_log_model(conn, model, model_name = "MY_MODEL",
                    input_cols = list(x = "double"),
                    output_cols = list(prediction = "double"),
                    conda_deps = c("r-base>=4.1", "rpy2>=3.5",
                                   "numpy<2.0", "r-forecast"))
```

Supported model types

Any R model that can be serialised with `saveRDS()` works, including:

- `lm()`, `glm()` (base R)
- `randomForest::randomForest()`
- `xgboost::xgb.train()`
- `ranger::ranger()`
- `forecast::auto.arima()`, `forecast::ets()`
- `tidymodels` workflows
- Custom S3/S4 model objects

The only requirement is that the model has a `predict()` method (or you provide custom prediction code via `predict_body`).